

I am completing my Software Engineering Lab-4 individual assignment.

Student Details

- Name: DHANUSH K S
- SRN: PES1UG24CS148
- Section: C
- Assigned repository:
https://github.com/SETAPESU26/21_checkers

Show more ▾



👍 You're at the **final submission/organization stage**. Since the coding, testing, and BEFORE/AFTER videos are already done, you mainly need to arrange everything cleanly.

1. Create your Lab-4 folder

Use this structure:

```
Lab-4/  
|  
├─ BEFORE_VIDEO.mp4  
├─ AFTER_VIDEO.mp4  
├─ Updated_Code/  
|   └─ [your project source files]  
|   └─ README.md  
└─ Chat_History.pdf
```

2. What each item should contain

BEFORE_VIDEO.mp4

- Shows the original/broken Checkers game.
- Keep the original video exactly as recorded.

AFTER_VIDEO.mp4

- Shows your corrected implementation.
- Ideally demonstrates the important features:
 - Capture
 - Forced capture
 - Multiple capture
 - King movement
 - Promotion
 - Win/no-move detection
 - Move feedback

Updated_Code/

- Put the **final working version** of your code here.

- Don't include unnecessary things such as:
 - .git folder
 - virtual environments
 - cache folders
 - temporary files
 - screenshots that aren't required

Chat_History.pdf

- Your Vibe Coding conversation showing the prompts and implementation process.
- Since the assignment specifically asks for chat history, this is useful evidence of how you developed the changes.

3. Important GitHub requirement

Your final code should be pushed to **your personal repository**, not the assigned repository.

Your assigned repository is:

SETAPESU26/21_checkers

So make sure your submission demonstrates:

Assigned repository → your work → your personal repository

And **do not create a PR** to SETAPESU26/21_checkers .

4. Final folder should look like this

```

Lab-4
├──
├── BEFORE_VIDEO.mp4
├── AFTER_VIDEO.mp4
├── Chat_History.pdf
├──
├── Updated_Code
│   ├── main.py
│   ├── ...
│   └── README.md
└──
    
```

If your Checkers project has a different set of source files, keep the **complete working project structure** rather than putting everything into one file.

5. Before submitting, do this final check

Requirement	Status
Repository cloned/forked	✓
README understood	✓
Original code tested	✓
BEFORE video	✓
Capture fixed	✓

Requirement	Status
Win/no-move detection	✓
Forced captures	✓
Multiple captures	✓
King movement	✓
Promotion	✓
Move-level feedback	✓
Major functionality tested	✓
AFTER video	✓
Code pushed to personal GitHub	✗
No PR to assigned repo	✓
Chat history PDF	✗
Lab-4 folder organized	✗

So you don't need to redo the coding work. Your remaining job is essentially GitHub verification + file organization + submission.

If you want, we can do the remaining part step-by-step starting with creating the Lab-4 folder and placing the exact files inside it.



If this is meant to be **TASK 2**, keep it exactly as a separate task in your chat-history record.

TASK 2 — Correct Capture Behavior

Vibe Coding Prompt:

"Fix the capture functionality in this Checkers game. When a player makes a valid capture move, the opponent's piece that was jumped over must be removed from the board, while the moving piece should remain at the destination. Do not change unrelated functionality."

Purpose:

Fix the bug where the opponent's piece remains on the board after a valid capture.

Expected result:

- The capture move is accepted.
- The opponent's jumped piece disappears.
- The player's piece moves to the destination.
- Other game functionality remains unchanged.

Testing:

Perform a valid capture and verify that the captured opponent piece is removed from the board.



"Implement the remaining Checkers rules required by the README: forced captures, multiple captures in the same turn, king movement in both directions, and correct promotion. Make sure captured pieces are removed and that a player must continue a multiple-capture sequence with the same piece when another capture is available."

TASK 3

TASK 3 — Implement Remaining Checkers Rules

Vibe Coding Prompt:

"Implement the remaining Checkers rules required by the README: forced captures, multiple captures in the same turn, king movement in both directions, and correct promotion. Make sure captured pieces are removed and that a player must continue a multiple-capture sequence with the same piece when another capture is available."

Purpose:

Implement the remaining core Checkers gameplay rules specified in the README.

Expected result:

- **Forced captures** — If a capture is available, the player must make a capture instead of a normal move.
- **Multiple captures** — After a capture, if the same piece can capture again, the player must continue with that piece.
- **King movement** — Kings can move and capture in both forward and backward directions.
- **Promotion** — A normal piece becomes a king when it reaches the opponent's back row.
- **Capture removal** — Every captured opponent piece is removed correctly.
- Turn changes only after the complete capture sequence is finished.



"Add concise player-facing feedback after every accepted move. For a normal move, show the player and source/destination coordinates. For a capture, indicate that a piece was captured. For promotion, indicate that the piece became a king. During multiple captures, provide feedback for each accepted capture. Do not print unnecessary extra messages." TASK 4

TASK 4 — Add Move-Level Feedback**Vibe Coding Prompt:**

"Add concise player-facing feedback after every accepted move. For a normal move, show the player and source/destination coordinates. For a capture, indicate that a piece was captured. For promotion, indicate that the piece became a king. During multiple captures, provide feedback for each accepted capture. Do not print unnecessary extra messages."

Purpose:

Provide clear, concise feedback to the player after every valid move.

Expected result:

- **Normal move:** Displays the player and source/destination coordinates.
- **Capture:** Indicates that an opponent piece was captured.
- **Promotion:** Indicates that the piece was promoted to a king.
- **Multiple captures:** Provides feedback after each individual capture.
- **No unnecessary messages:** Only relevant move feedback is displayed.



ChatGPT can make mistakes. Check important info.